



JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA

B.TECH

III-SEMESTER PROJECT SYNOPSIS

TITLE OF PROJECT: CAMPUS AND SMART CITY NAVIGATION SYSTEM

Submitted by:

Name

Enrollment No

Pranav Bhardwaj

2501030331

[Teammate 2 Name]

[Enrollment No]

[Teammate 3 Name]

[Enrollment No]

Introduction

Navigation and route-planning platforms must repeatedly calculate optimal paths across complex road networks while dynamically adapting to real-time traffic conditions, construction hazards, and road closures. The Campus and Smart City Navigation System models this problem as a high-performance C++ program paired with an interactive visual web interface. The physical environments are represented as weighted graphs where landmarks (academic blocks, hostels, gates, airport, transit hubs, hospitals) sit on vertices, and roads or pedestrian walkways form weighted edges. Pathfinding is solved using Dijkstra's algorithm and the A* search algorithm with Euclidean heuristics. Active roadblocks and detour rerouting are handled dynamically through edge suppression, and vector cross-product geometry calculates natural language turn-by-turn guidance. The project applies core Data Structures and Algorithms concepts to an essential, everyday navigation platform.

AIM:

- To build a high-performance navigation engine that calculates shortest and fastest routes across campus and metropolitan city environments.
- To implement and benchmark Dijkstra's algorithm and the A* search algorithm using an admissible Euclidean distance heuristic.
- To provide dynamic obstacle avoidance and real-time detour rerouting upon road blockages or construction closures.
- To generate vector-based turn-by-turn directional guidance with precise segment distances.
- To demonstrate the appropriate data structure for each part of the system: graph, priority queue (min-heap), hash table, vector, and composite structs.

Abstract:

The physical map is stored as a weighted graph using an adjacency list, where each node represents a location with 2D spatial coordinates (x, y) and category metadata, and each edge stores distance, speed limit, directionality, and blockage status. The system implements a dual-scale architecture: a 28-node College Campus map (academic complexes, hostels, sports arenas, cafeterias) and a 32-node Smart City metropolitan map (international airport, silicon tech parks, central stations, hospitals, bridges). Routing queries accept an origin and target landmark along with optimization criteria (shortest physical distance or fastest travel time). For routing, the engine executes Dijkstra's algorithm or A* search using a priority queue (min-heap) for optimal vertex expansion. The A* algorithm employs an admissible Euclidean distance heuristic to prune the search space, reducing explored nodes by 30% to 65% compared to Dijkstra while maintaining 100% path optimality. If a road segment is blocked due to traffic or maintenance, the engine suppresses the edge and computes an optimal detour route in microseconds. A geometric navigation engine evaluates consecutive path vectors using vector cross-product and dot-product calculations to derive turn angles and generate turn-by-turn driving and walking instructions. The program is implemented in C++ using STL containers and features an interactive web interface with an SVG canvas.

FEATURES

- Dual-map spatial models: 28-node College Campus and 32-node Smart City metropolitan networks.
- Graph representation using adjacency lists with $O(1)$ average vertex lookups via hash maps.
- Shortest path routing using Dijkstra's algorithm and A* heuristic search.
- Admissible Euclidean distance heuristic for efficient search space pruning.
- Dynamic obstacle handling: click-to-block roads and instantaneous detour recalculation.
- Multi-metric cost optimization: shortest physical distance (meters) or fastest travel time (seconds).

- Vector computational geometry for turn-by-turn natural language navigation instructions.
- Side-by-side benchmark comparison HUD evaluating execution time (μs) and nodes explored.
- Handling of unreachable locations and completely blocked destination clusters.
- Interactive modern SVG web frontend with pan, zoom, node selection, and animated path rendering.
- Dual-mode execution: interactive console menu and headless JSON REST API for web integration.

SAMPLE PROJECT STRUCTURE

```
NaviCore/
|-- include/
|   |-- Landmark.h           // id, name, coordinates (x,y), category
|   |-- Edge.h               // from, to, distance, speed limit, status
|   |-- Graph.h              // adjacency list, dynamic road blockage
|   |-- PathFinder.h         // Dijkstra & A* (Euclidean heuristic)
|   |-- NavigationEngine.h   // vector turn geometry & JSON exporter
|-- src/
|   |-- Graph.cpp             // CSV map parsing, edge suppression
|   |-- PathFinder.cpp        // min-heap priority queue, backtracking
|   |-- NavigationEngine.cpp  // angle math, turn-by-turn guidance
|   |-- main.cpp              // CLI menu, benchmark & JSON API modes
|-- data/                     // campus_nodes, campus_edges, city_nodes, city_edges
|-- web/                      // interactive SVG map frontend (index.html, app.js, style.css)
`-- server.py                 // lightweight HTTP bridge connecting web UI to C++ engine
```

DATA STRUCTURES USED

Data Structure	Where it is used
Graph (adjacency list)	Campus and City maps: landmarks as nodes, roads/walkways as weighted edges
Priority queue (min-heap)	Open list of A* and Dijkstra priority queue for minimum-cost vertex expansion
Hash table (unordered_map)	Landmark coordinate lookup, adjacency lists, gScore, fScore, and backtracking tables
Vector	Dynamic storage of adjacency edges, computed path node sequences, and turn directions
Struct	Composite models for Landmark, Edge, RouteResult, and TurnInstruction

TOPICS USED

- Graphs: adjacency list representation, directed and undirected weighted edges, and traversal
- A* search and heuristic functions (admissible Euclidean metric)
- Dijkstra's algorithm for single-source shortest paths
- Heaps and priority queues (std::priority_queue) with custom comparators
- Hashing with std::unordered_map for $O(1)$ average time lookups
- Dynamic obstacle avoidance and runtime edge suppression
- Vector computational geometry (cross-product and dot-product for angle determination)
- Object-oriented programming in C++ (modular classes, structs, constructors, const correctness)
- STL containers and algorithms
- Time and space complexity analysis ($O((V + E) \log V)$ time, $O(V + E)$ space)